

Explainable AI for Network Intrusion Detection

Using SHAP to understand why an ML-based IDS blocks a flow

Mini Research Problem
Artificial Intelligence – Advanced Topics in AI & ML

Code: github.com/Wangzx9908/ai-hw-SUMMER-2026-MRP

Motivation

An ML-based intrusion detection system only outputs *normal* or *attack*. A network engineer cannot act on that: was the flow blocked because of abnormal TCP flags, or because the payload was empty? Without a reason, nobody trusts the system enough to let it drop real traffic.

Main idea

Train a Random Forest on the NSL-KDD flow dataset to classify traffic into *normal* / *DoS* / *Probe* / *R2L* / *U2R*, then apply **SHAP** to see which of the 41 flow features drove each decision – globally (per attack type) and locally (a single flow).

Results

- 74.3 % accuracy on the official test set (DoS recall 77 %, Probe 65 %).
- SHAP shows the model relies on exactly the features a network engineer would use: DoS → `serror_rate`, `count`; Probe → `src_bytes=0`, `diff_srv_rate`.
- So the AI decision is consistent with classic protocol analysis, and every block can now be explained in one sentence.

Literature review

Why black-box IDS models are a problem

- **Sommer & Paxson (IEEE S&P 2010)** – ML-based intrusion detection suffers from a *semantic gap*: an anomaly score is not an actionable finding, operators need the reason.
- **Arp et al. (USENIX Security 2022)** – security models often learn spurious shortcuts; inspecting the learned features is necessary.

The two standard explanation methods

- **LIME** (Ribeiro et al., KDD 2016) – fits a simple linear model around one prediction. Fast, works on any model, but it samples randomly, so repeated runs can give different answers.
- **SHAP** (Lundberg & Lee, NeurIPS 2017) – distributes the prediction among the features using Shapley values from game theory. Slower, but consistent and additive; *TreeSHAP* (Lundberg et al., Nature MI 2020) computes it exactly for tree models, which is why we use it here.

Applied to intrusion detection

- **Wang et al. (IEEE Access 2020)** – SHAP-based framework for IDS on NSL-KDD and UNSW-NB15; the reference for this project.
- **Mane & Rao (2021)** – SHAP and LIME on NSL-KDD deep models.
- **Warnecke et al. (IEEE EuroS&P 2020)** – compares explanation methods on security tasks and finds SHAP the most reliable.

Approach

1. Dataset – NSL-KDD (unb.ca/cic/datasets/nsl.html)

- Classic benchmark for flow-based intrusion detection (Tavallaee et al., CISDA 2009).
- 125 973 training flows / 22 544 test flows, **41 features** extracted from PCAP: byte counts, TCP flags, connection rates, host counters.
- Attack names grouped into 4 families: DoS, Probe, R2L, U2R (+ normal). The official test split deliberately contains attack types absent from training.

2. Model – Random Forest ([scikit-learn](https://scikit-learn.org/))

- 100 trees, balanced class weights. Chosen because *TreeSHAP* can explain it exactly and cheaply – no approximation.

3. Explainability – SHAP (github.com/shap/shap)

- `shap.TreeExplainer` on 1667 test flows → one SHAP value per (flow, feature, class).
- **Global explanation:** average $|\text{SHAP}|$ per feature, separately for DoS and Probe → what defines each attack type for the model.
- **Local explanation:** waterfall plot of one flow → why *this* packet flow was blocked.

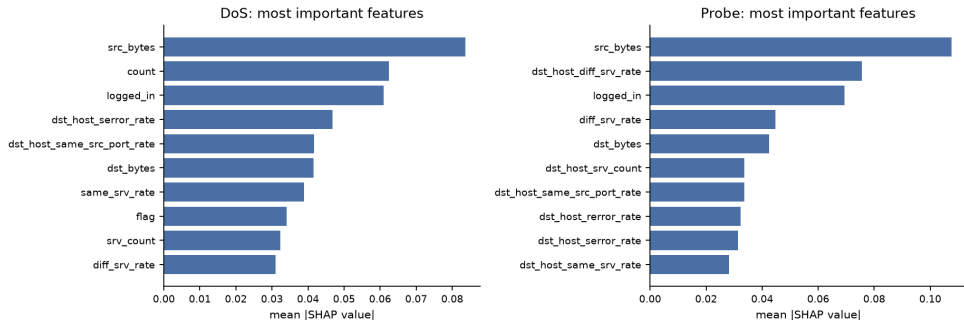
Everything runs from one script: `python3 src/nids_xai.py` (about 1 minute).

Step 1: the detector works, but says nothing

Class	Recall	Support
normal	97 %	9 711
DoS	77 %	7 460
Probe	65 %	2 421
R2L	0 %	2 885
U2R	6 %	67
Overall accuracy: 74.3 %		

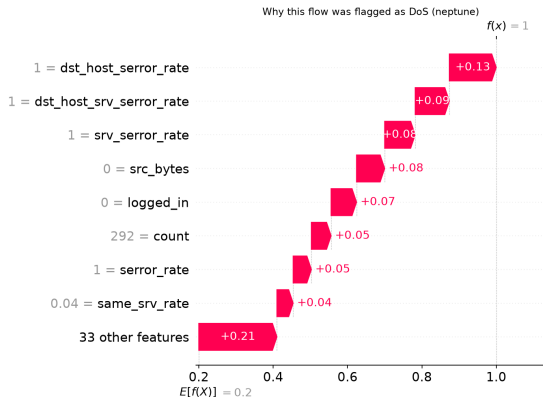
- Volumetric attacks (DoS, Probe) are detected reasonably well; R2L and U2R almost never are.
- But this table tells an engineer *nothing* about the individual decision that just dropped a customer's connection.
- That is the gap SHAP fills in the next slides: the same model, now with a reason attached to every verdict.

Step 2: global explanation – what defines each attack



- **DoS:** `src_bytes` (almost no payload), `count` (many connections in a short window) and `dst_host_serror_rate` (failed / half-open TCP handshakes) – the classic **SYN flood** pattern.
- **Probe:** `src_bytes=0` combined with high `dst_host_diff_srv_rate` and `diff_srv_rate` – empty connections spread over many different services, i.e. a **port scan**.
- The model relies on the same indicators an engineer would check manually.

Step 3: local explanation – one blocked flow (DoS)



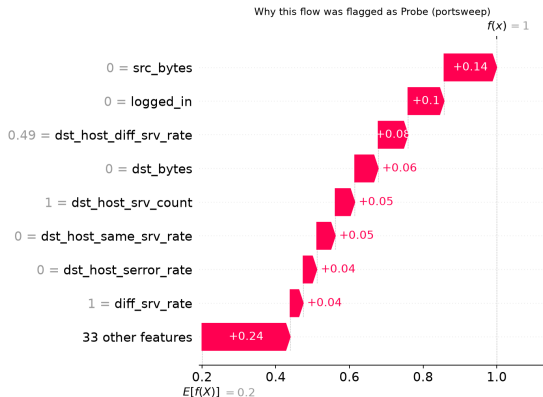
A single neptune (SYN flood) flow, classified as DoS with $p = 1.00$. The bar chart starts from the average prediction $E[f(x)] = 0.20$ and shows how each feature pushed it to 1.00.

Turned into a sentence an operator can read:

Blocked as **DoS** because the SYN error rate to this host is **100 %** (`serror_rate=1`), the flow carries **0 bytes** of payload, and there were **292** connections in the time window.

All three facts are directly verifiable in the flow record – the engineer can confirm or reject the verdict in seconds.

Step 3: local explanation – one blocked flow (Port scan)



A portsweep flow, classified as Probe with $p = 1.00$.

Flagged as **Probe** because the flow sends **0 bytes**, the session was **never logged in**, and **49%** of connections to this host went to *different* services ($\text{dst_host_diff_srv_rate}=0.49$).

Note how different this reason is from the DoS one: no error rates, no volume – instead, service fan-out. The same model produces a different, attack-specific explanation, which is exactly what makes the output useful.

Conclusion

What was done

- Trained a Random Forest IDS on NSL-KDD (74.3 % on the official test split).
- Explained it with SHAP at both levels: global feature importance per attack type, and per-flow waterfall plots.

What we learned

- The model's reasoning agrees with classic protocol analysis (SYN error rates for DoS, service fan-out for port scans), which is what makes it trustworthy enough to deploy.
- Interpretability doubles as a debugging tool (Lecture 09): the features SHAP highlights also explain *why* R2L attacks are missed – they leave no volumetric trace in a single flow.

Limitations

- NSL-KDD describes 1999-era traffic; CICIDS2017 would be the modern follow-up.
- TreeSHAP is exact only for tree models; a deep IDS needs the slower KernelSHAP or gradient-based methods.

Links and references

Code – one script reproduces every figure in this deck

github.com/Wangzx9908/ai-hw-SUMMER-2026-MRP

Data and tools

- NSL-KDD: unb.ca/cic/datasets/nsl.html
- CICIDS2017: unb.ca/cic/datasets/ids-2017.html
- SHAP: github.com/shap/shap LIME: github.com/marcotcr/lime
- scikit-learn: scikit-learn.org
- Interpretable ML book: christophm.github.io/interpretable-ml-book

References

- Lundberg & Lee, *A Unified Approach to Interpreting Model Predictions*, NeurIPS 2017.
- Lundberg et al., *From local explanations to global understanding with explainable AI for trees*, Nature Machine Intelligence 2020.
- Ribeiro et al., *"Why Should I Trust You?"*, KDD 2016.
- Sommer & Paxson, *Outside the Closed World*, IEEE S&P 2010.
- Warnecke et al., *Evaluating Explanation Methods for Deep Learning in Security*, IEEE EuroS&P 2020.
- Wang et al., *An Explainable ML Framework for Intrusion Detection Systems*, IEEE Access 2020.
- Tavallaee et al., *A Detailed Analysis of the KDD CUP 99 Data Set*, CISDA 2009.

Thank you

github.com/Wangzx9908/ai-hw-SUMMER-2026-MRP